

Formal Methods in Software Engineering

Assignment 2 (Spin and Model-Checking)

(Due on Thu 26th Feb 2026)

1. Consider the pairs of LTL assertions below. For each pair either give an informal proof that they are equivalent, or a counter-example model (an infinite sequence of propositional valuations) that shows they are not equivalent. Your counter-example should be in the form of an ultimately periodic word of the form $u \cdot (v^\omega)$, where u and v are finite sequences of propositional valuations over the propositions $\{p, q, r\}$.
 - (a) $G(Fp)$ and $F(Gp)$.
 - (b) $p\mathcal{U}q$ and $(p \vee q) \wedge G((p \wedge \neg q) \Rightarrow X(p \vee q))$.
2. Suppose you are given a deterministic Büchi automaton $\mathcal{A}$ over an alphabet Σ . Show how you can give a (possibly non-deterministic) Büchi automaton $\mathcal{B}$ that accepts the complement of its language.
3. Consider the Promela model below.

```
byte x = 1;

proctype counter() {
  do
    :: x <= 1 -> x = x + 1;
    :: x >= 1 -> x = x - 1;
  od
}

init {
  run counter();
}
```

- (a) Describe the transition system that Spin compiles this model into. You may choose to ignore the intermediate states Spin uses to move to a satisfied guard. Show only the reachable states.
 - (b) Does this model satisfy the property “ $(x = 1) \Rightarrow F(x = 2)$ ”? Justify your answer.
 - (c) Construct a state-based Büchi automaton (by hand if you like) for the *negation* of this formula, over the propositions p representing $x = 1$, and q representing $x = 2$.
 - (d) Construct the product of the transition system of the model and the Büchi automaton for the formula above. Describe the counterexample path if any.
4. Construct the formula automaton for the LTL formula $true \mathcal{U} p$, using the algorithmic construction described in class. You could treat $true$ as $p \vee \neg p$.

5. This problem asks you to use the Spin model-checker to reason about the correctness of a distributed data structure.

Consider the design of a counter Convergent Replicated Data Type (CRDT) in the accompanying tutorial on CRDTs. The counter is similar to the Op-based increment-only counter from the tutorial that supports increment, and read operations, except that now we have an additional decrement operation. The CRDT comprises multiple replicas of the counter state which is simply an integer initialized to 0. Each replica allows local update operations of increment and decrement, and a local query operation of read. In an Op-based CRDT, for each update operation at a replica, the replica must broadcast the name of the operation and the timestamp of the operation (op, t) to all other replicas. On receiving such a message, the receiving replica applies the corresponding operation on its local state.

Each replica can be said to have an *abstract* state, which is the set of (operation, timestamp) pairs corresponding to update operations it has seen so far, either directly (through a local operation) or indirectly (through a message about a remote operation). The CRDT is said to be *strongly eventually consistent* if whenever the abstract states of two replicas match, so do their local states.

Your job is to use Spin to check if the given CRDT satisfies strong eventual consistency in the presence of (a) lossless channels modelling a reliable network (b) lossy channels modelling a network that can lose messages (c) duplicating channels that models a network that can duplicate messages, and (d) reordering channels that model a network that can reorder messages (i.e. a message sent from a replica can overtake messages sent from the same replica).

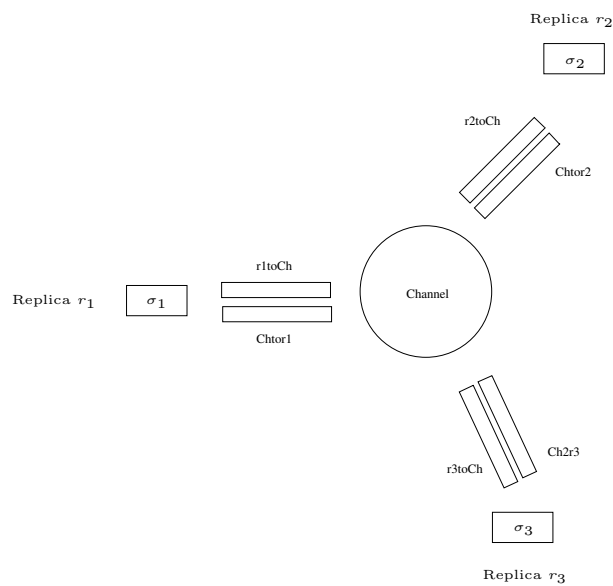
You can assume that there are only three replicas and a total of 5 operations across replicas. You can assume that there is a global “time” variable that the replicas use to get a timestamp and increment atomically. The `time` variable is initialized to 0.

Model the abstract state of each replica as a timestamp-indexed array of size 5.

Phrase an LTL property “sec” that asserts the property of strong eventual consistency.

For case (a) above you can model the network using 6 channels “r1tor2”, “r1tor3”, “r2tor1”, “r2tor3”, “r3tor1” and “r3tor2”, each of capacity 5 messages.

For case (b)–(d) model the network as a separate process that communicates with the replicas using two channels each. This is shown in the schematic below:



Submit your Promela models in separate files called “counter-crdt-direct.pml”, “counter-crdt-lossy.pml”, “counter-crdt-duplicate.pml”, and “counter-crdt-reorder.pml”. Describe your findings in an accompanying text file called “results.txt”